

Machine Learning 4

Sequence Models

Example:

- speech recognition:  $\rightarrow$ "quick brown fox jumped over the fence."
- Music Generation: $\phi \rightarrow$ 
- Sentiment Classification: "There is nothing to like in the movie" $\rightarrow$ ★★★★★
- DNA sequence Analysis: AGCCTTTAGGGCATG ... $\rightarrow$ AGCCTTT AGGG CATG ...
- Machine Translation: "xyz of different language" $\rightarrow$ "abc in another language".
- Video Activity Recognition.

Note: Can all be modelled as supervised learning problems.

Notation

Example: Normal Entity Recognition (Find proper nouns)

$$x : \underbrace{\text{Harry}}_{x^{(1)}} \underbrace{\text{Potter}}_{x^{(2)}} \text{ \& Hermione Granger invented a new spell.}$$
$$y : \underbrace{1}_{y^{(1)}} \underbrace{1}_{y^{(2)}} 0 1 1 0 0 0 0$$

Note: In the above example 1 tells us that something is indeed a proper noun (or part of), and 0 tells us that it's not (not a part of either).

$$T_X = 9 \text{ (length of input),}$$

$$T_Y = 9$$

Note: $x^{<1>}$ continues to $x^{<t>}$, t here implies that it's a temporal sequence.

$x^{(i)<t>}$: t^{th} training element in the sequence of training example i

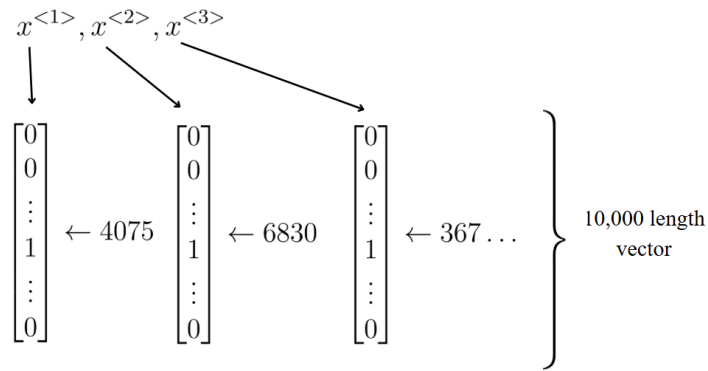
$T_X^{(i)}$: input sequence length of training example i

$T_Y^{(i)}$: output sequence length of training example j

We can come up with a vocabulary (10,000 word dictionary),

a	1
$Aaron$	2
$\vdots$	$\vdots$
and	367
$\vdots$	$\vdots$
$Harry$	4075
$\vdots$	$\vdots$
$Potter$	6830
$\vdots$	$\vdots$
$Zulu$	10,000

We can use "one-hot encoding" to represent the words



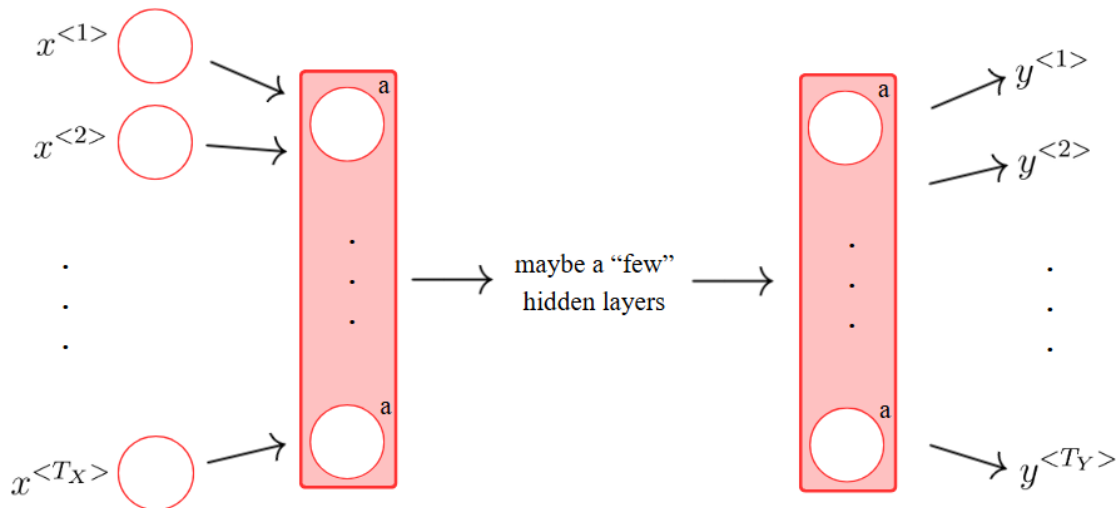
Note: Where $x^{<1>}$ corresponds to the word "Harry", $x^{<2>}$ corresponds to the word "Potter" and so on, ...

Note: Here, $x^{<t>}$ is a "one-hot" vector

learn $X \rightarrow Y$ (Supervised Learning Problem)

Note: This example assumes we will not come across words that aren't in the dictionary.

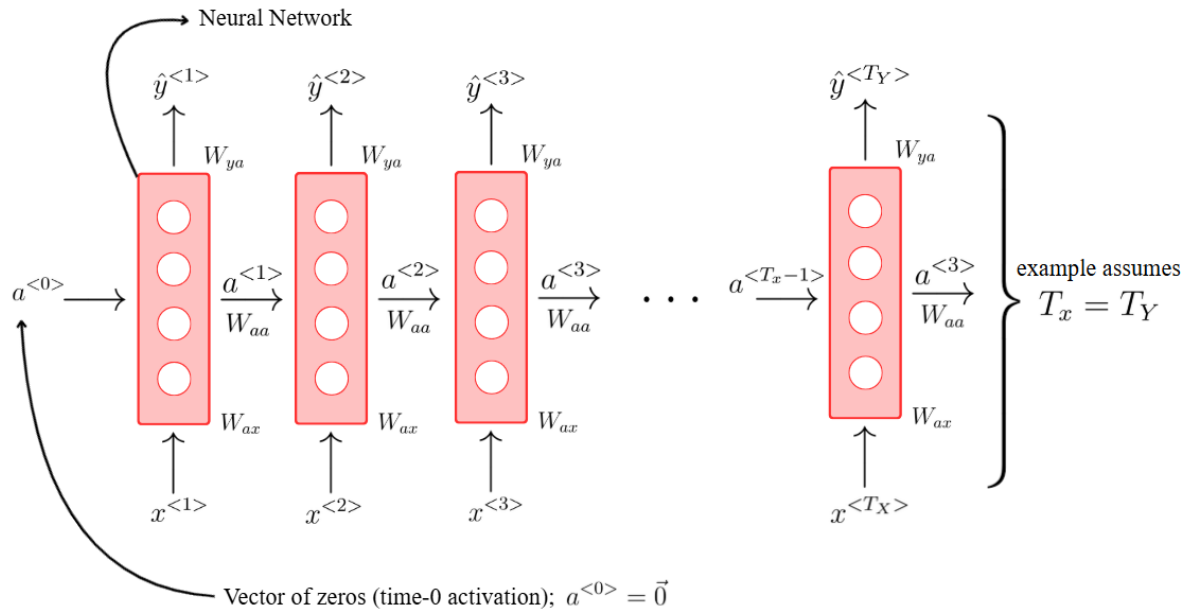
Why not a standard Neural Network ?



Problems:

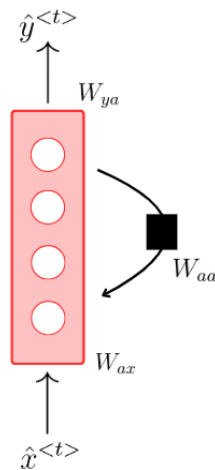
- Inputs & Outputs can be different lengths and sizes in different examples
- Doesn't share features learned across different positions of text
 - May learn "Harry" is a name but only at a specific position, i.e., "Harry" is only a name if it occurs at (say) $x^{<1>}$, but in practice it could be present at $x^{<t>}$ for any $t < T_X$.
- Since each input-vector is of length-10,000, The input layer can become stupidly large (80,000 or something) many neurons.

RNNs



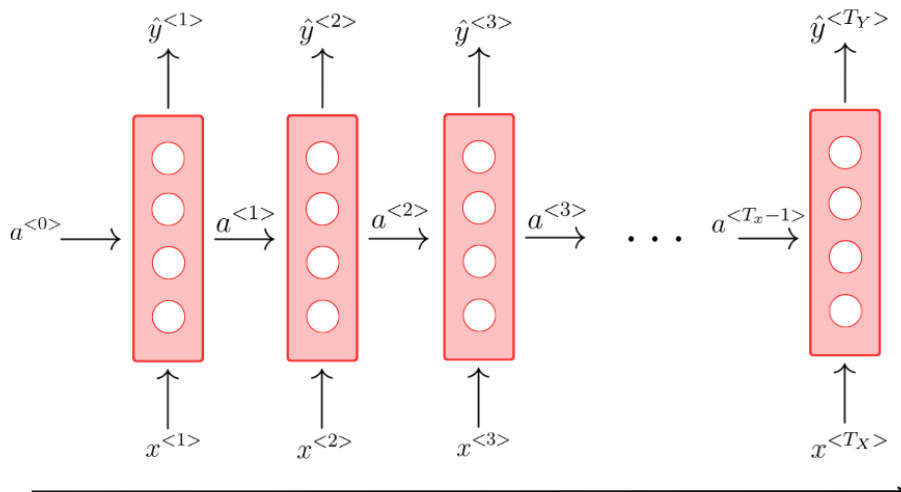
Note: Here W_{aa} , W_{ya} & W_{ax} are all parameters for each Neural Network.

This can be more succinctly denoted as,



Note: A problem with RNNs is that since they are sequential. Anything learned in $x^{<1>}$, ... or earlier layers cannot be improved by what exists in later layers. This problem is usually solved using BRNNs (Bidirectional-RNNs).

Forward Propagation in a RNN

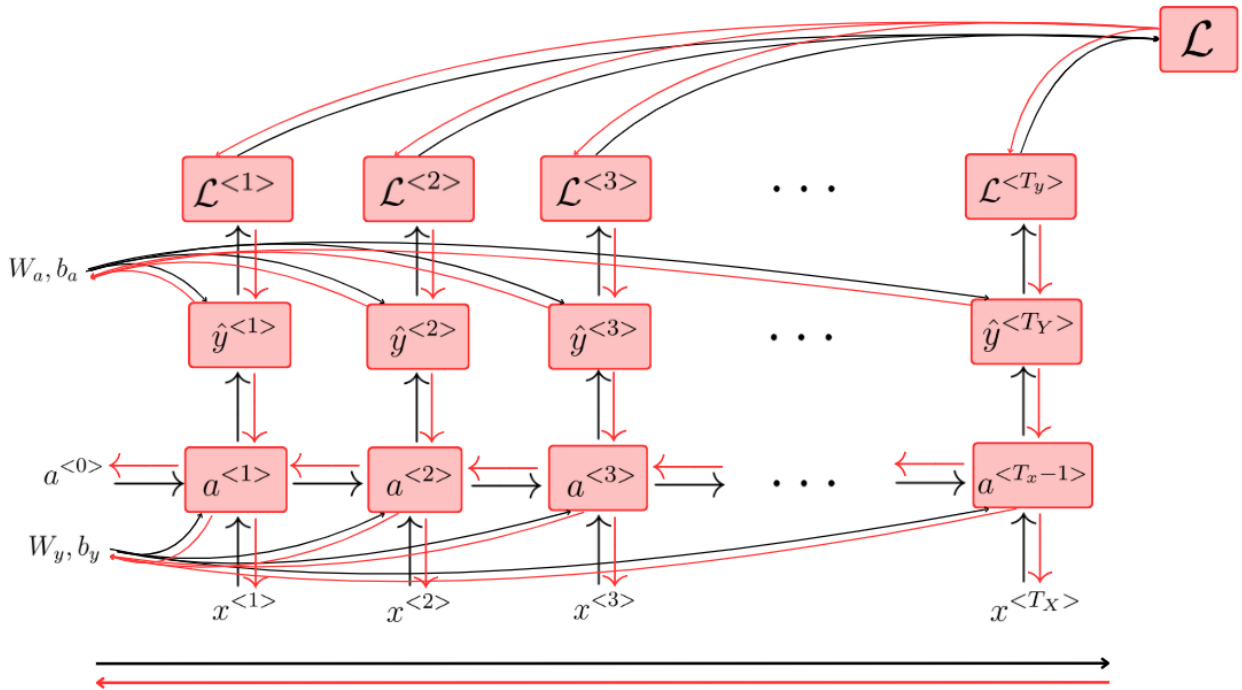


$$\begin{aligned}
a^{<1>} &= g(W_{aa}a^{<0>} + W_{ax}X^{<1>} + b_a) \leftarrow \text{tan h / ReLU} \\
\hat{y}^{<1>} &= g(W_{ya}a^{<1>}b_y) \leftarrow \text{sigmoid / softmax (depends on output)} \\
&\rightarrow \begin{cases} a^{<t>} = g(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \\ \hat{y}^{<t>} = g(W_{ya}a^{<t>} + b_y) \end{cases} \\
&\Rightarrow a^{<t>} = g(W_a[a^{<t-1>}, x^{<t>}] + b_a)
\end{aligned}$$

where, $[W_{aa} \mid W_{ax}] = W_a$. This is just for notational simplicity,

$$\begin{aligned}
[a^{<t-1>}, x^{<t>}] &= \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix} \text{ upside down stacked} \\
\rightarrow [W_{aa}|W_{ax}] \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix} &= W_{aa}a^{<t-1>} + W_{ax}x^{<t>}
\end{aligned}$$

Back-propagation in a RNN (forward + back-propagation)



Standard Logistic Regression Loss / Cross Entropy Loss

$$\begin{aligned}
\mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>}) &= -y^{<t>} \log \hat{y}^{<t>} - (1 - y^{<t>}) \log(1 - \hat{y}^{<t>}) \\
\rightarrow \mathcal{L}(\hat{\dagger}, \dagger) &= \sum_{t=1}^{T_y} \mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>})
\end{aligned}$$

Note: aka "Back-propagation through time".

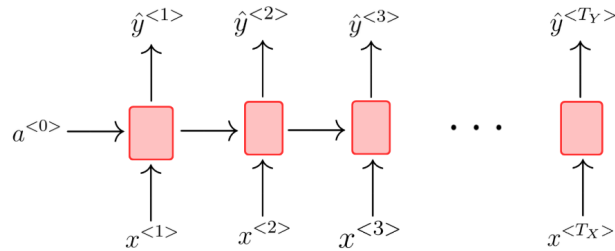
Different types of RNNs

Many-to-many

If we take,

$$T_x = T_y$$

we get,

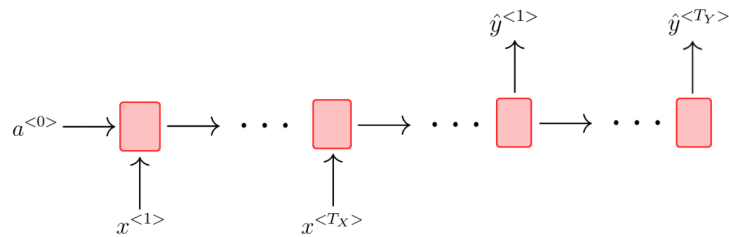


Many-to-many

Generally,

$$T_x \neq T_y$$

The part from left until the NN that accepts input from $x^{<T_x>}$ is known as the encoder whereas the part on its right is called the decoder.

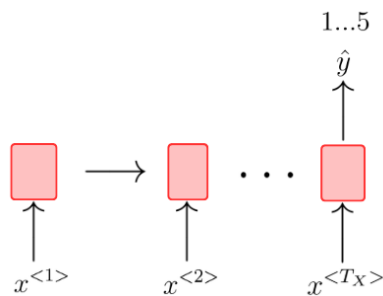


Many-to-one

Example: Sentiment Classification we could have,

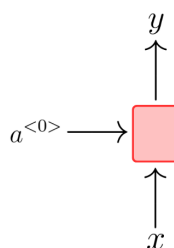
$x = \text{text}$

$y = 0/1$



One-to-one

Would just be a simple Neural Network.

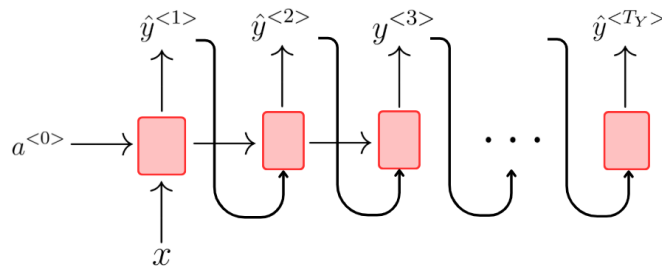


One-to-many

Example: Music Generation

$$x \rightarrow y^{<1>} y^{<2>} \dots y^{<T_y>}$$

Note: x could be ϕ



What is language modelling (with a RNN)

Example:

Consider speech recognition,

$$\begin{cases} \text{"The apple \& pair salad"} \\ \text{"The apple \& peer salad"} \end{cases}$$

Note: These sound exactly the same but the difference will be informed by a language model.

$$\begin{cases} P(\text{"..."}) = 3.2 * 10^{-13} \\ P(\text{"..."}) = 5.7 * 10^{-10} \end{cases}$$

Note: The latter has greater probability.

Goal:

$$P(\text{sentence}) = ?, \quad P(y^{<1>}, y^{<2>}, y^{<3>}, \dots, y^{<T_y>})$$

Training set: A large corpus of English text.

Example:

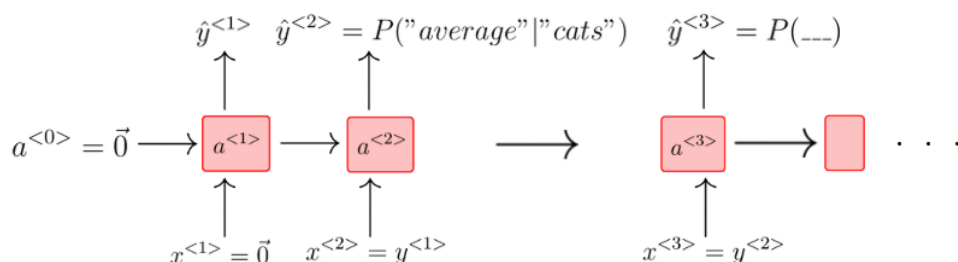
$$\begin{matrix} \text{Cats} & \text{average} & 15 & \text{hours} & \text{of} & \text{sleep} & \text{a} & \text{day} & . \\ y^{<1>} & y^{<2>} & y^{<3>} & \dots & & & & & <EOS> \end{matrix}$$

Note: common practice to include an end of sentence character, so model can learn that as well $<EOS>$. Other punctuation marks could also be modelled.

Note: "The Egyptian Mam is a breed of cat", whenever new proper nouns / words are encountered, we can model them as $<UNK>$.

Example:

$$\rightarrow P(a), P(aaron), \dots, P(cats), \dots, P(Tulu), \dots, P(<EOS>)$$



We can also get probability of getting the 3-word sentence above,

$$= P(y^{<1>}) \cdot P(y^{<2>} | y^{<1>}) \cdot P(y^{<3>} | y^{<1>}, y^{<2>})$$

To revise,

$$\mathcal{L}(\hat{y}^{<t>}, y^{<t>}) = - \sum_i y_i^{<t>} \log \hat{y}_i^{<t>} \leftarrow \text{Softmax Loss Function}$$

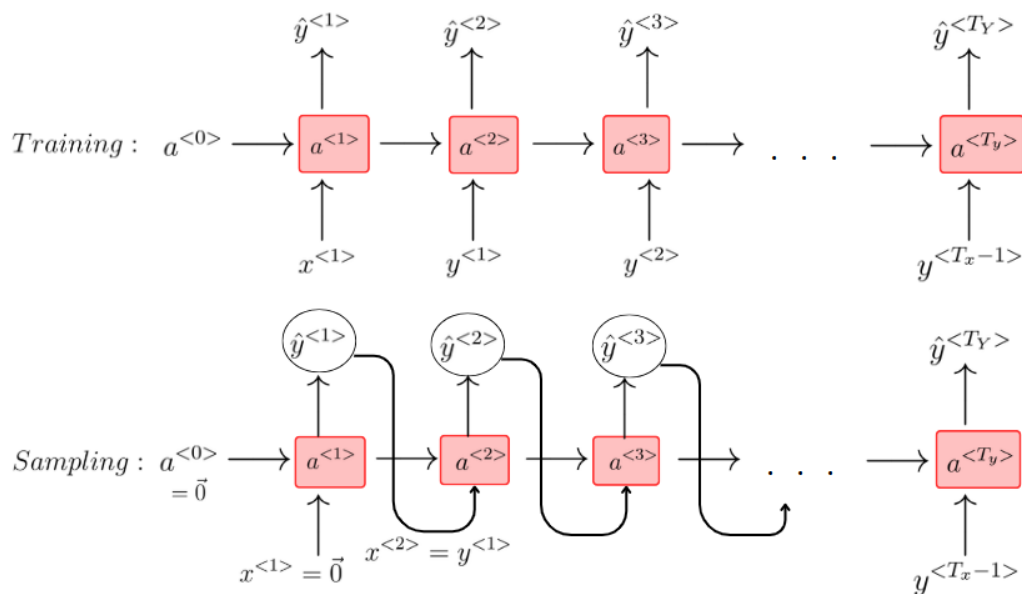
and, the overall loss is,

$$\mathcal{L} = \sum_i \mathcal{L}^{<t>}(\hat{y}^{<t>}, y^{<t>})$$

Sampling Normal Sequences

Sampling a sequence from a trained RNN,

Example:



Character-Level Language Model

Vocabulary = [a, aaron, ..., zulu, <UNK>]

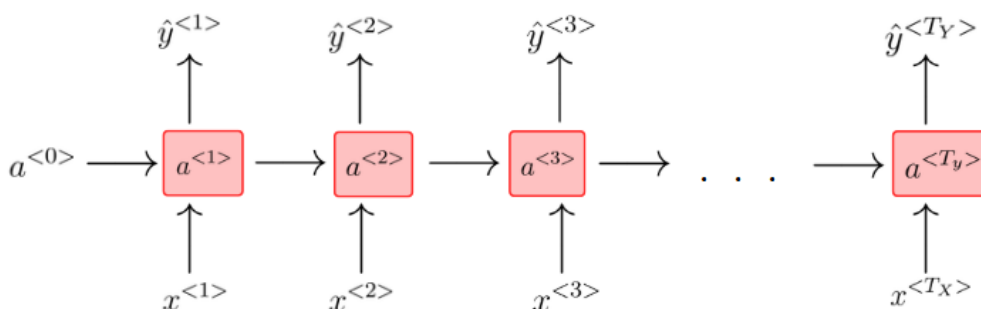
Vocabulary = [a, b, c, ..., z, ., , , ; , 0, ..., 9, A, ..., Z]

instead of word-level vocabulary.

Note: $y^{<1>}, y^{<2>}, y^{<3>}, \dots$ will become characters not words.

- **Disadvantage:** Large Arguments
- **Advantage:** Will end up having unknown words with non-0 probabilities in such as proper nouns.

Vanishing-Gradients with RNNs



Example:

The cat, which already ate ..., was full (singular)

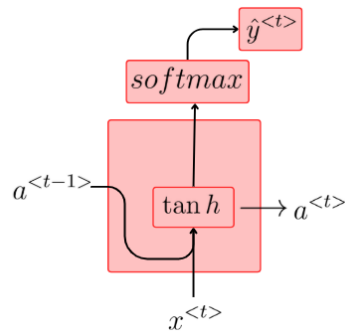
The cats, which already ate ..., were full (plural)

Note: Long-term dependencies occur when early words in a sentence can influence later words. During backpropagation, this information can be lost as gradients become increasingly small when propagated through many time steps.

Note: We can also encounter “exploding gradients”, where gradients become exponentially large. This is usually easier to detect because the model’s updates become extremely large or unstable. Gradient clipping provides a robust way to mitigate this problem.

GRU: Gated-Recurrent-Unit

RNN units visualisation of hidden layer



$$a^{<t>} = \underbrace{g(W_a[a^{<t-1>}, x^{<t>}] + b_a)}_{\text{if tanh}}$$

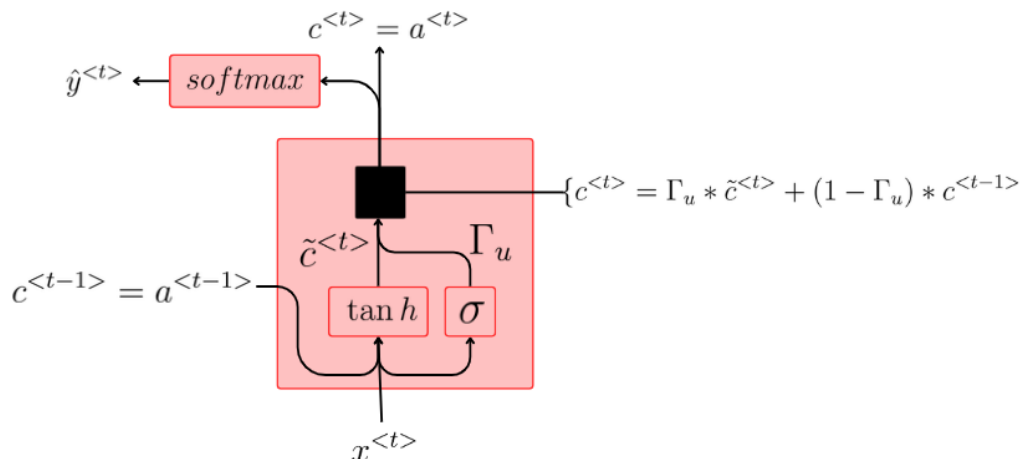
GRU (Simplified)

The cat, which already ate ..., was full

- c = memory-cell
- $c^{<t>} = a^{<t>}$
- $\tilde{c}^{<t>} = \tanh(W_c[c^{<t-1>}, x^{<t>}] + b_c)$
- $\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$
- $c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$

Note: Γ_u , u is the update term (always either 0 or 1). and Γ is the gate. If gate is 0, don't update value hang onto it.

GRU-unit (simplified)



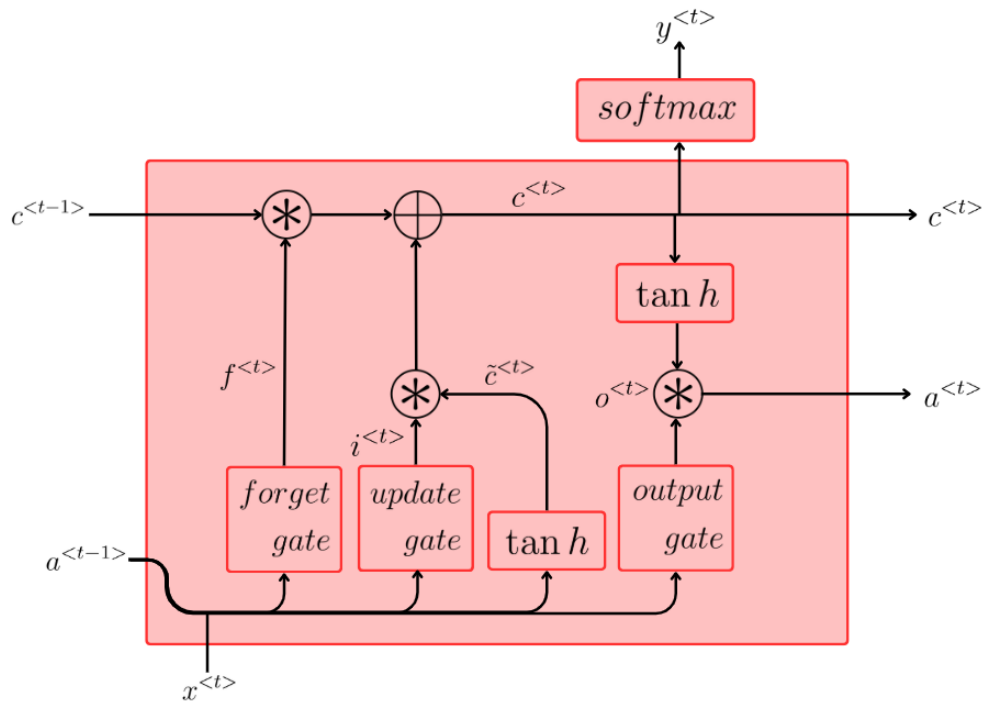
Full-GRU Unit

- $\tilde{c}^{<t>} = \tan h(W_c[\Gamma_r^* c^{<t-1>}, x^{<t>}] + b_c)$
- $\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$
- $\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$
- $c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}, a^{<t>} = c^{<t>}$

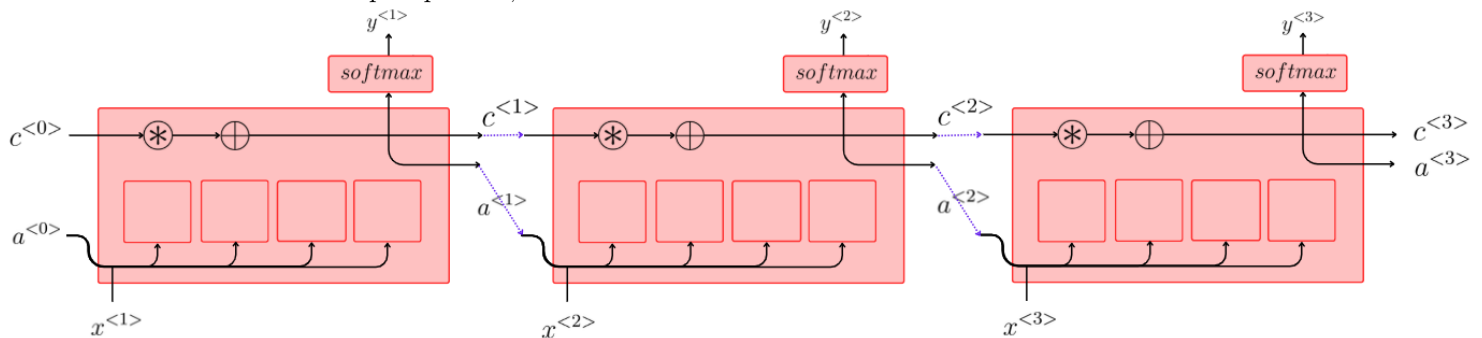
LSTM

- even more general than GRU.
- requires more theory.
- $\tilde{c}^{<t>} = \tan h(W_c[a^{<t-1>}, x^{<t>}] + b_c)$
- $\Gamma_u = \sigma(W_u[\underbrace{a^{<t-1>}, x^{<t>}}_{\text{update}}] + b_u)$ - update
- $\Gamma_f = \sigma(W_f[\underbrace{a^{<t-1>}, x^{<t>}}_{\text{forget}}] + b_f)$ - forget
- $\Gamma_o = \sigma(W_o[\underbrace{a^{<t-1>}, x^{<t>}}_{\text{output}}] + b_o)$ - output
- $c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}, a^{<t>} = \Gamma_o * \tan h(c^{<t>})$

Note: Gate values may depend on previous cell values. Sometimes people sneak in their $c^{<t-1>}$ values as well: "peephole connection" (common variation)



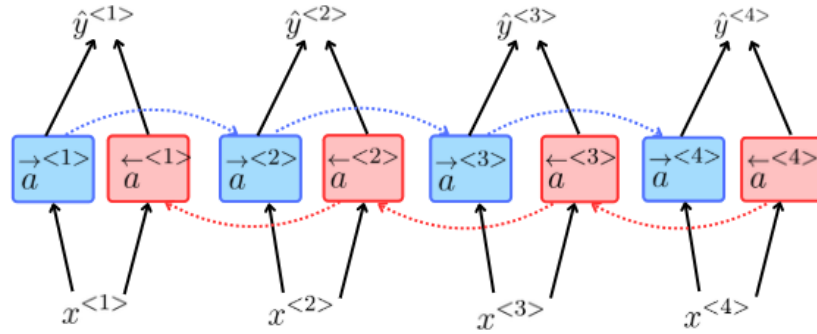
These units would be stacked up in parallel,



Bidirectional-RNN (BRNN)

Note: Helps get information from the future.

Doing this we get an Acyclic graph,



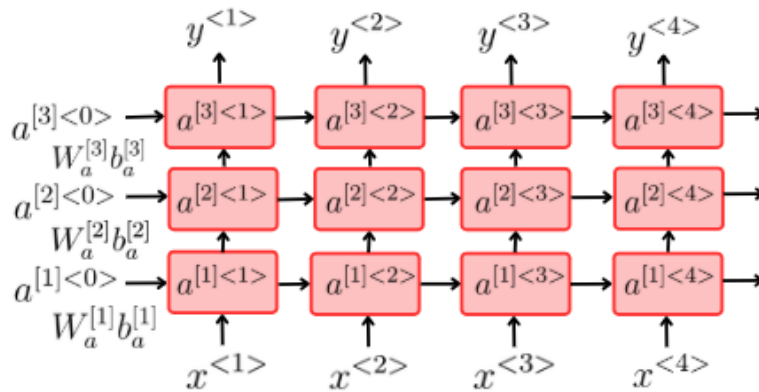
where,

$$\hat{y}^{<t>} = g \left(W_y \left[\vec{a}^{<t>}, \overleftarrow{a}^{<t>} \right] + b_y \right)$$

Note: Here, each block could be either a GRU or an LSTM.

Deep RNNs

Let $a^{[l]<t>}$ denote activation at layer l, time t.



Note: Can have different depth.

Note: Each of the blocks (like before), can be either GRU or LSTM.

Example:

$$a^{[2]<3>} = g \left(W_a^{[2]} \left[a^{[2]<2>}, a^{[1]<3>} \right] + b_a^{[2]} \right)$$